

Simulating Dipole-Based EEG Signals: From Laplace's Equation to Machine Learning Localization

Matthew Lindley

PHY380 Spring 2025

Abstract

Electrostatics underlies much of our understanding of neural activity and field-based diagnostics. Inspired by a suite of open-source Java applications built on the Open Source Physics (OSP) framework, this project explores electric field behavior using numerical solutions to Laplace's equations. By extending these models to simulate scalp voltage measurements from interior neural sources, this work lays a foundation for electroencephalography (EEG) simulation. The paper outlines the logic of the core simulation methods, establishes a computational grid for modeling diffusion from sources and dipoles, and proposes further steps toward implementing both forward and inverse EEG modeling on a 2D lattice, culminating in a machine learning approach for predicting dipole positions from propagating internal sources of potential difference.

I. Introduction

Electric and magnetic fields govern the behavior of physical and biological systems, from atomic interactions to brain signal propagation. In the human cortex, EEG signals originate from synchronized firings of neuron populations. These activities create **dipole-like electric fields** which propagate through layers of tissue and are measurable at the scalp (Rayi & Murr, 2023). Simulating this process numerically allows us to test source localization methods that are otherwise impossible to validate directly (He et al., 2023).

This project was inspired by a trio of Java-based Open Source Physics (OSP) applications: `ElectricFieldApp.java`, `FieldLineApp.java`, and `LaplaceApp.java`. These tools visualize how point charges generate potentials, how field lines emerge from these charges, and how grid-based relaxation can model potential distributions. Leveraging the insight gained from these apps, I developed `EEGSimulatorApp.java` and `CornerElectrodes.java` to simulate the propagation of voltages through a grid representing the anatomy of the skull. Using these, I explored how placing a dipole affects readings at corner electrodes, followed by a larger simulation in `EEGSimulator2.java` that generates many such dipoles and exports a dataset for machine learning classification. This dataset is used by `predictdipole.py` to test whether it's feasible to predict the location of the dipole using just electrode readings.

This project builds on foundational concepts of Laplace's heat equation (3Blue1Brown, 2019), which illustrates the connection between continuous differential equations like and their discretized, numerically solved counterparts. These ideas provide the stepping stones for our simulations. Finally, by framing the inverse EEG problem as a classification task, we seek to test the limits of spatial resolution using only four electrodes.

II. Theoretical Background and Tools

Electrostatic fields are governed by Laplace's equation, written as $\nabla^2 V = 0$, which applies in regions where no free charge is present. It implies that the electric potential at any point is equal to the average of its surrounding points, leading to a smooth equilibrium distribution over space. In contrast, Poisson's equation accounts for the presence of free charge. This is essential for modeling biological systems like EEG, where active neurons act as localized sources of current. To simulate these behaviors computationally, continuous equations must be transformed into discrete approximations. The 3Blue1Brown heat equation video (3Blue1Brown, 2019) provided valuable intuition for this process, showing how time-dependent diffusion settles into steady-state behavior described by Laplace's equation. In our simulation, Jacobi relaxation numerically replicates this behavior by repeatedly updating each point on a 2D grid to the average of its four neighbors. Over time, this mimics the diffusion of potential through space, ultimately reaching a state where no further change occurs. This method is ideal for modeling electric field propagation in a controlled environment like a simplified 2D cross-section of the head.

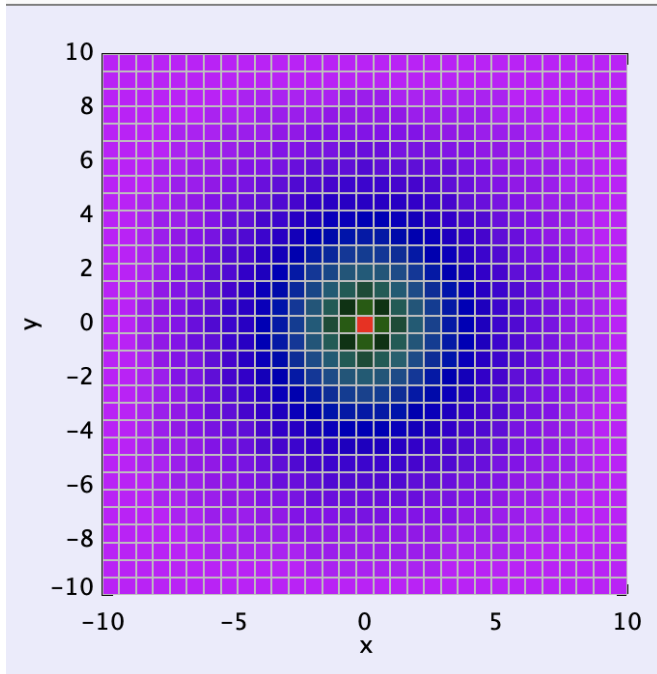


Figure 1: Sample Laplace Diffusion Matrix

To gain intuition and build confidence in these numerical methods, several open-source Java tools from the Open Source Physics (OSP) framework were used. `ElectricFieldApp.java` simulates the electric field vectors generated by point charges, helping visualize how dipoles produce directionally dependent fields. `FieldLineApp.java` traces electric field lines using numerical integration, showing how fields flow from positive to negative sources—directly analogous to dipole activity in the cortex. `LaplaceApp.java` numerically solves Laplace’s equation using Jacobi relaxation, visualizing how potential evolves from fixed voltage conditions on the grid. Together, these tools form the theoretical and practical backbone for simulating EEG-like systems, demonstrating how physical laws can be mapped onto a digital grid and used to explore real-world biological modeling.

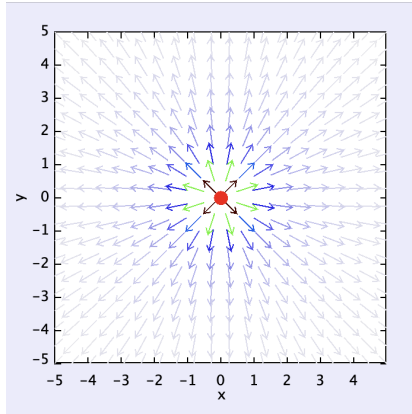
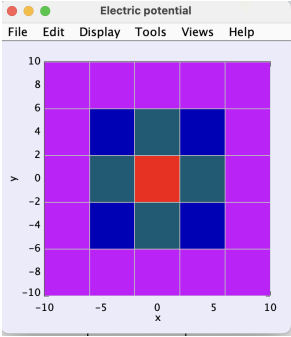
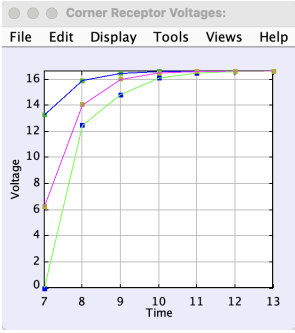
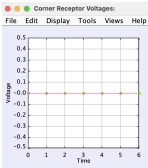
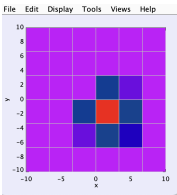
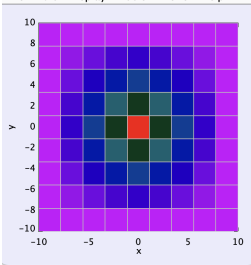
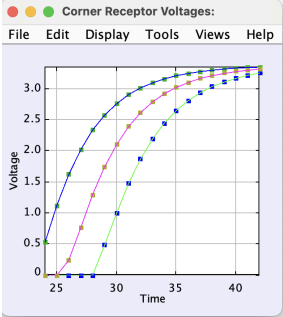


Figure 2: Sample output of OSP's ElectricFieldApp.java

III. Methods

As the initial java file in this project, CornerElectrodes.java was developed to validate whether voltages from a central source could be meaningfully detected at peripheral electrode positions. It is based on the LaplaceApp.java from OSP. A simplified 9x9 grid was used, with a fixed +100V source at the center. The surrounding boundary was defined as insulating (non-conductive) to represent the boundary of the scalp, and relaxation was allowed to proceed until equilibrium. At each time step, voltage values at the four corners (representing electrodes) were recorded. This served as a proof-of-concept: electric potential diffuses through the grid in a measurable and structured way, and peripheral voltages vary as a function of source proximity and symmetry. The resulting table below illustrates the voltage evolution at each corner electrode. In some runs, overlap occurs due to symmetric values at opposing corners, but the data clearly demonstrates that the electrode readings respond to the central source. This validation step confirmed that even a small number of electrode measurements can provide valuable insight into the internal voltage configuration.

Electrodes distance away from center (d_c)	GridSize (s) ~ s x s	Electric Potential Grid	Electrode Voltages
1	5		
2	5		 Receptors are on insulating boundaries.
3	6		indexOutOfBounds error. Receptors expand outside of the boundary.
3	9		

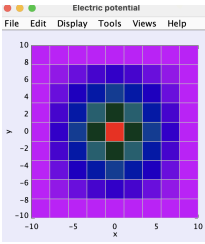
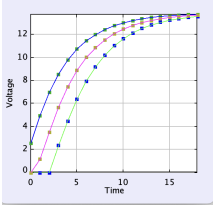
2	9		 Receptors are closer than previous run $\Rightarrow$ higher voltage values.
---	---	--	--

Table 1: Corner Electrode Efficacy

Once it was established that the corner electrodes could provide meaningful data, the initial EEGSimulatorApp.java was created to build on the prior test by simulating a more complex concept – a dipole: one +100V and one -100V charge placed adjacently inside the grid. To be clear, in modeling EEG activity, we create dipoles because synchronized neuronal firing produces dipole-like electric fields, which then propagate through the brain and are measurable at the scalp (Rayi & Murr, 2023). This dipole induces directional field lines and complex voltage propagation. The grid is relaxed using Jacobi iteration, and the four corner electrodes measure potential. In Figure 3 below, a run of a 15x15 grid is shown to completion. Its electrostatic potential diffuses towards the corners, which you must imagine have the electrodes. Furthermore, contrasting the EEGSimulatorApp, which has a dipole moment, to the CornerElectrodes.java program, you can see a disparity of trends between corner receptor voltages and time. See Figure 3A.

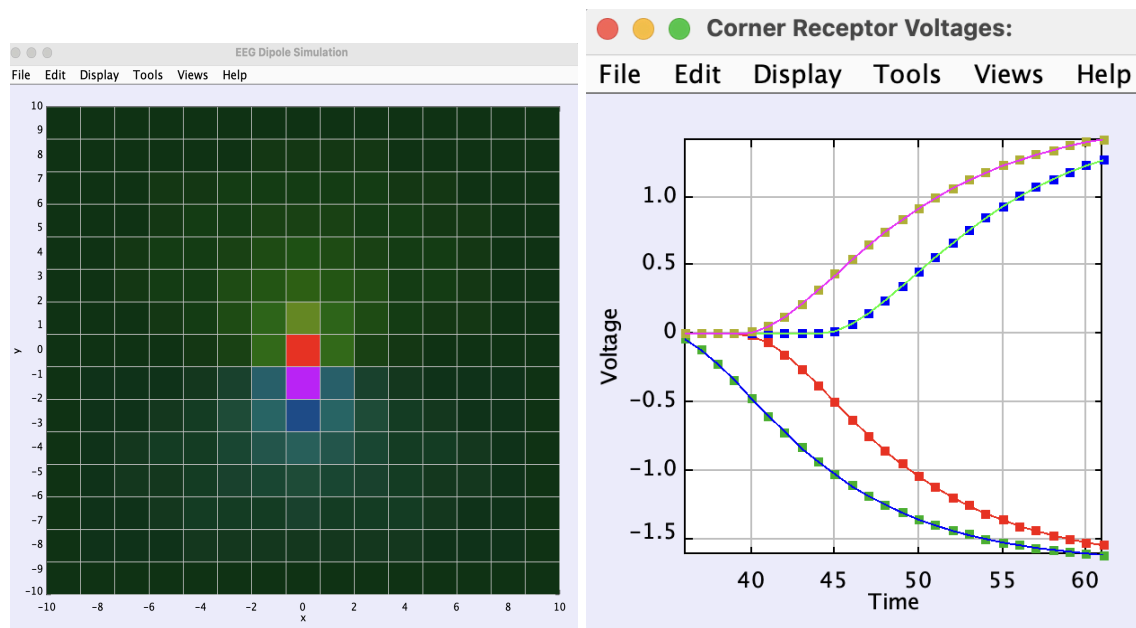


Figure 3: *Dipole simulated in center of grid.* **Figure 3A:** *Four corner electrodes readings. Red = Top Left Electrode | Blue = Top Right | Green = Bottom Left | Magenta = Bottom Right*

Now that it was confirmed we could obtain disparate data from differently positioned electrodes, it became increasingly of interest to see if a machine learning model could predict where these electrical signals were propagating from. If we could deduce the position of propagating signals with high accuracy, then maybe in a much more complex setup we could predict what regions of the brain are being activated during episodes like seizures, psychosis, or even during education and learning. Thus, the more advanced version of EEGSimulator2App.java was created and placed 100 random dipole configurations on the grid (ensuring they are not on the boundary or at electrode sites). Each configuration consists of a +100V and -100V pair, placed adjacently. After relaxing the grid, electrode readings from Upper Left (UL), UR, LL, and LR are stored in a row along with the dipole positions. The data is exported to a CSV file, which has been attached on coursesite. Finally, predictdipole.py is a Python script that loads the CSV file and splits the

data into training and test sets. A machine learning classifier (e.g., RandomForest or LogisticRegression) is trained to predict the dipole's location based on the electrode readings. Since the dipole is a pair of positions, the model predicts both as a combined output (e.g., a multi-label classification task).

IV. Inverse Problem Justification

The inverse EEG problem centers on determining the internal source configuration that gives rise to voltage measurements recorded at the scalp. This challenge is fundamentally ill-posed, meaning that multiple internal configurations can produce the same surface readings, making exact reconstruction inherently uncertain. As a result, inverse modeling often requires mathematical constraints, anatomical priors, or regularization techniques to guide solutions toward physiological plausibility. In clinical and research settings, solving the inverse problem is critical for localizing brain activity, such as identifying epileptic foci or decoding neural signals for brain-computer interfaces. In this project, a brute-force simulation strategy was combined with machine learning to explore whether spatial patterns in voltage readings from just four corner electrodes could be used to infer dipole locations within a 2D grid. Although the model's exact prediction accuracy was modest, it demonstrated the ability to consistently approximate general regions of dipole activity, suggesting that sparse data can still encode meaningful spatial information. **Visualizations further revealed that predicted dipole positions often clustered near the true coordinates, even when they were not exact.** This outcome reinforces the idea that simplified inverse approaches can still uncover structure in the data. While the current implementation is limited by the resolution of the grid and the number of electrodes, it lays the groundwork for future studies incorporating dense electrode arrays, heterogeneous tissue conductivities, and more advanced learning models. Simulations like these offer a valuable

means of testing and validating inverse problem strategies in a controlled, interpretable environment, before transitioning to the complexities of real-world EEG data.

V. Results

The CornerElectrodes.java model demonstrated that voltage readings change significantly depending on the proximity and orientation of a dipole source. Higher voltage values at the electrodes were consistently observed when the dipole was located closer to a given corner, validating that the diffusion pattern captured meaningful spatial information. This proof-of-concept established that corner electrodes are sensitive to central sources, even in a simplified Laplacian system. The EEGSimulator2App.java application extended this approach by generating **100 randomized dipole configurations** on a **16×16 grid**, simulating the resulting voltage field using Jacobi relaxation, and storing the four corner electrode readings along with the true dipole coordinates. This dataset was exported as a CSV file to serve as training data for machine learning. The predictdipole.py script used a multi-output random forest regressor to predict the dipole's location based solely on the four electrode voltages. While the model was able to approximate general dipole regions, exact positional accuracy was limited: the model achieved a 5% exact match rate when predicting all four dipole coordinates correctly. On average, predictions deviated by approximately *3.6 grid units from the ground truth*. Visualizations, including a heatmap and scatter plots of predicted versus actual positions, confirmed that the model learned rough spatial relationships but struggled with precision at this resolution and input sparsity. These results highlight the complexity of the inverse problem and the importance of further refinement—through either increased electrode count, more sophisticated models, or the incorporation of anatomical priors—before reliable localization can be achieved.

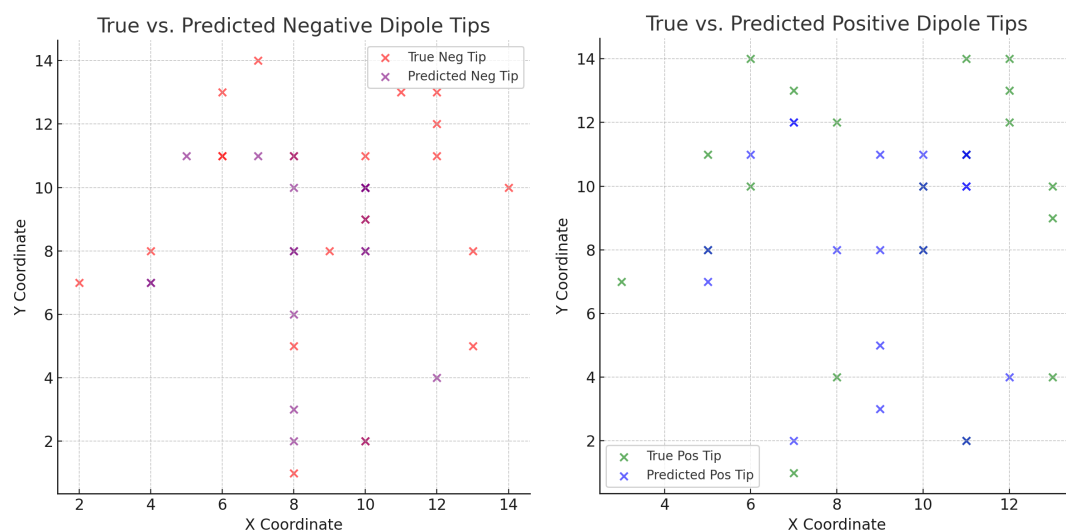


Fig. 4: Scatter Plots comparing True vs. Predicted Dipole Coordinates

To be clear, in further assessment of model performance, scatter plots were generated comparing the true versus predicted dipole coordinates – think the popular board game *Battleship*. In the first plot, green dots represent the actual positions of the positive dipole tips, while blue Xs mark the model's predictions. The second plot similarly displays red dots for the actual negative dipole tips and purple Xs for their predicted counterparts. While the model rarely achieved exact matches, the predictions frequently clustered near the true values, indicating spatial awareness. These visualizations support the conclusion that the model captures directional electric field behavior across the 2D grid, even with a limited number of input electrodes.

VI. Future Directions and Conclusion

This project combines classical electrodynamics, numerical modeling, and machine learning to simulate EEG activity and explore how to localize internal dipole sources. Moving forward, one improvement would be to allow for variable conductance in the simulation. This would shift from using Laplace's equation to Poisson's equation, making the model more realistic by accounting for differences in how the brain, skull, and scalp conduct electricity. Another step would be to include anatomical information from brain scans, helping guide where dipoles can be placed and improving the accuracy of predictions. The simulation could also be made dynamic by introducing time-varying sources, allowing us to model how neuron activity changes over time. Expanding from a 2D grid to a 3D volume would make the setup more lifelike and allow for more detailed modeling of voltage flow in the brain. On the machine learning side, using deep learning models could improve predictions by capturing more complex relationships between electrode readings and source positions. Increasing the number of electrodes from four to a larger array would also improve accuracy and better reflect real EEG systems. This project shows that physical models can be used to create synthetic data for testing source localization methods. While the accuracy of the current model was limited, it could still predict general areas where dipoles were located. This suggests that even simple setups can carry useful information. With further development, this approach could become a powerful way to test and improve inverse EEG methods before applying them to real patient data.

Works Cited

3Blue1Brown. (2019). Solving the Heat Equation [YouTube Video].

<https://www.youtube.com/watch?v=GzbKb59my3U>

Barzegaran, E., Bosse, S., Kohler, P. J., & Norcia, A. M. (2019). EEGSourceSim: A framework for realistic simulation of EEG scalp data using MRI-based forward models and biologically plausible signals and noise. *Journal of Neuroscience Methods*, 328, 108377.

<https://doi.org/10.1016/j.jneumeth.2019.108377>

Christian, W., & Belloni, M. (2004). Open Source Physics. *American Journal of Physics*, 72(6), 847–853. <https://doi.org/10.1119/1.1632486>

Rayi, A., & Murr, N. (2023). Electroencephalogram. *StatPearls*.

<https://www.ncbi.nlm.nih.gov/books/NBK538500/>

Glomb, K., et al. (2022). Computational Models in Electroencephalography. *Brain Topography*, 35(1), 15–30. <https://doi.org/10.1007/s10548-021-00872-5>

He, H., et al. (2023). Inverse Problem for M/EEG Source Localization: A Review. *IEEE Transactions on Biomedical Engineering*. <https://doi.org/10.1109/TBME.2023.3271009>

CornerElectrodes.java

```
/*
 * Open Source Physics software is free software as described near the bottom of this code file.
 *
 * For additional information and documentation on Open Source Physics please see:
 * <http://www.opensourcephysics.org/>
 */

package org.opensourcephysics.sip.ch10;

import java.awt.event.*;
import org.opensourcephysics.controls.*;
import org.opensourcephysics.display.*;
import org.opensourcephysics.display2d.*;
import org.opensourcephysics.frames.*;

public class LaplaceApp extends AbstractSimulation implements InteractiveMouseHandler {
    Scalar2DFrame frame = new Scalar2DFrame("x", "y", "Electric potential");
    PlotFrame graphFrame = new PlotFrame("Time", "Voltage", "Corner Receptors");

    boolean[][] isConductor;
    double[][] potential;
    double maximumError;
    int gridSize;
    int distanceOfElectrodes;
    int timeStep = 0;

    double[] topLeftHistory = new double[1000];
    double[] topRightHistory = new double[1000];
    double[] bottomLeftHistory = new double[1000];
    double[] bottomRightHistory = new double[1000];

    public LaplaceApp() {
        frame.setInteractiveMouseHandler(this);
    }

    public void initialize() {
        maximumError = control.getDouble("maximum error");
        gridSize = control.getInt("size");
        distanceOfElectrodes = control.getInt("distance of electrodes");
        initArrays();
        frame.setVisible(true);
        frame.showDataTable(true);
    }
}
```

```

graphFrame.setVisible(true);
graphFrame.setConnected(true);
graphFrame.setMarkerSize(0, 2);
graphFrame.setMarkerSize(1, 2);
graphFrame.setMarkerSize(2, 2);
graphFrame.setMarkerSize(3, 2);
}

public void initArrays() {
    isConductor = new boolean[gridSize][gridSize];
    potential = new double[gridSize][gridSize];
    frame.setPaletteType(ColorMapper.DUALSHADE);

    for (int i = 0; i < gridSize; i++) {
        for (int j = 0; j < gridSize; j++) {
            potential[i][j] = 0;
            isConductor[i][j] = false;
        }
    }

    int center = gridSize / 2;
    potential[center][center] = 100;
    isConductor[center][center] = true;

    frame.setAll(potential);
}

public void doStep() {
    double error = 0;
    for (int i = 1; i < gridSize - 1; i++) {
        for (int j = 1; j < gridSize - 1; j++) {
            if (!isConductor[i][j]) {
                double v = (potential[i - 1][j] + potential[i + 1][j] +
                    potential[i][j - 1] + potential[i][j + 1]) / 4.0;
                double dv = potential[i][j] - v;
                error = Math.max(error, Math.abs(dv));
                potential[i][j] = v;
            }
        }
    }

    // Update display
    frame.setAll(potential);
}

```



```

int cx = gridSize / 2;
int dist = distanceOfElectrodes;

double topLeft    = potential[cx - dist][cx + dist];
double topRight   = potential[cx + dist][cx + dist];
double bottomLeft = potential[cx - dist][cx - dist];
double bottomRight = potential[cx + dist][cx - dist];

topLeftHistory[timeStep] = topLeft;
topRightHistory[timeStep] = topRight;
bottomLeftHistory[timeStep] = bottomLeft;
bottomRightHistory[timeStep] = bottomRight;

graphFrame.append(0, timeStep, topLeft);
graphFrame.append(1, timeStep, topRight);
graphFrame.append(2, timeStep, bottomLeft);
graphFrame.append(3, timeStep, bottomRight);

// Set line colors
graphFrame.setLineColor(0, java.awt.Color.RED);
graphFrame.setLineColor(1, java.awt.Color.BLUE);
graphFrame.setLineColor(2, java.awt.Color.GREEN);
graphFrame.setLineColor(3, java.awt.Color.MAGENTA);

// Set legend labels properly
graphFrame.getDataset(0).setName("Top Left (RED)");
graphFrame.getDataset(1).setName("Top Right (BLUE)");
graphFrame.getDataset(2).setName("Bottom Left (GREEN)");
graphFrame.getDataset(3).setName("Bottom Right (MAGENTA)");
graphFrame.setTitle("Corner Receptor Voltages:\nRed = TL | Blue = TR | Green
= BL | Magenta = BR");

timeStep++;

if (error < maximumError) {
    animationThread = null;
    control.calculationDone("Computation done.");
}
}

public void reset() {
    control.setValue("maximum error", 0.1);
    control.setValue("size", 31);
    control.setValue("distance of electrodes", 0);

```

```

        initialize();
    }

    public void handleMouseAction(InteractivePanel panel, MouseEvent evt) {
        switch (panel.getMouseAction()) {
            case InteractivePanel.MOUSE_DRAGGED:
            case InteractivePanel.MOUSE_PRESSED:
                double x = panel.getMouseX();
                double y = panel.getMouseY();
                int i = frame.xToIndex(x);
                int j = frame.yToIndex(y);
                frame.setMessage("V=" + decimalFormat.format(potential[i][j]));
                break;
            case InteractivePanel.MOUSE_RELEASED:
                panel.setMessage(null);
                break;
        }
    }

    public static void main(String[] args) {
        SimulationControl.createApp(new LaplaceApp());
    }
}

```

EEGSimulatorApp.java

```

package org.opensourcephysics.sip.ch10;

import java.awt.event.*;
import org.opensourcephysics.controls.*;
import org.opensourcephysics.display.*;
import org.opensourcephysics.display2d.*;
import org.opensourcephysics.frames.*;

public class EEGSimulatorApp extends AbstractSimulation implements
InteractiveMouseHandler {
    Scalar2DFrame frame = new Scalar2DFrame("x", "y", "EEG Dipole Simulation");
    PlotFrame graphFrame = new PlotFrame("Time", "Voltage", "Corner Receptors");

    boolean[][] isConductor;
    double[][] potential;
    int gridSize;
    double maximumError;
    int timeStep = 0;
}

```

```

double[] topLeftHistory = new double[1000];
double[] topRightHistory = new double[1000];
double[] bottomLeftHistory = new double[1000];
double[] bottomRightHistory = new double[1000];

int posX, posY;

public EEGSimulatorApp() {
    frame.setInteractiveMouseHandler(this);
}

public void initialize() {
    maximumError = control.getDouble("maximum error");
    gridSize = control.getInt("size");
    initArrays();
    frame.setVisible(true);
    frame.showDataTable(true);

    graphFrame.setVisible(true);
    graphFrame.setConnected(true);
    graphFrame.setMarkerSize(0, 2);
    graphFrame.setMarkerSize(1, 2);
    graphFrame.setMarkerSize(2, 2);
    graphFrame.setMarkerSize(3, 2);
}

public void initArrays() {
    isConductor = new boolean[gridSize][gridSize];
    potential = new double[gridSize][gridSize];
    frame.setPaletteType(ColorMapper.DUALSHADE);

    for (int i = 0; i < gridSize; i++) {
        for (int j = 0; j < gridSize; j++) {
            potential[i][j] = 0;
            isConductor[i][j] = false;
        }
    }

    posX = gridSize / 2;
    posY = gridSize / 2;

    potential[posX][posY] = 100;
    potential[posX][posY + 1] = -100;

```

```

isConductor[posX][posY] = true;
isConductor[posX][posY + 1] = true;

frame.setAll(potential);
}

public void doStep() {
    double error = 0;
    for (int i = 1; i < gridSize - 1; i++) {
        for (int j = 1; j < gridSize - 1; j++) {
            if (!isConductor[i][j]) {
                double v = (potential[i - 1][j] + potential[i + 1][j] +
                    potential[i][j - 1] + potential[i][j + 1]) / 4.0;
                double dv = potential[i][j] - v;
                error = Math.max(error, Math.abs(dv));
                potential[i][j] = v;
            }
        }
    }
}

frame.setAll(potential);

int dist = gridSize / 2 - 2;
int cx = gridSize / 2;

double topLeft = potential[cx - dist][cx + dist];
double topRight = potential[cx + dist][cx + dist];
double bottomLeft = potential[cx - dist][cx - dist];
double bottomRight = potential[cx + dist][cx - dist];

topLeftHistory[timeStep] = topLeft;
topRightHistory[timeStep] = topRight;
bottomLeftHistory[timeStep] = bottomLeft;
bottomRightHistory[timeStep] = bottomRight;

graphFrame.append(0, timeStep, topLeft);
graphFrame.append(1, timeStep, topRight);
graphFrame.append(2, timeStep, bottomLeft);
graphFrame.append(3, timeStep, bottomRight);

graphFrame.setLineColor(0, java.awt.Color.RED);
graphFrame.setLineColor(1, java.awt.Color.BLUE);
graphFrame.setLineColor(2, java.awt.Color.GREEN);
graphFrame.setLineColor(3, java.awt.Color.MAGENTA);

```

```

graphFrame.getDataset(0).setName("Top Left (RED)");
graphFrame.getDataset(1).setName("Top Right (BLUE)");
graphFrame.getDataset(2).setName("Bottom Left (GREEN)");
graphFrame.getDataset(3).setName("Bottom Right (MAGENTA)");

graphFrame.setTitle("Corner Receptor Voltages:\nRed = TL | Blue = TR | Green = BL |
Magenta = BR");

timeStep++;

if (error < maximumError) {
    animationThread = null;
    control.calculationDone("Computation done.");
}
}

public void reset() {
    control.setValue("maximum error", 0.1);
    control.setValue("size", 31);
    initialize();
}

public void handleMouseAction(InteractivePanel panel, MouseEvent evt) {
    switch (panel.getMouseAction()) {
        case InteractivePanel.MOUSE_DRAGGED:
        case InteractivePanel.MOUSE_PRESSED:
            double x = panel.getMouseX();
            double y = panel.getMouseY();
            int i = frame.xToIndex(x);
            int j = frame.yToIndex(y);
            frame.setMessage("V=" + decimalFormat.format(potential[i][j]));
            break;
        case InteractivePanel.MOUSE_RELEASED:
            panel.setMessage(null);
            break;
    }
}

public static void main(String[] args) {
    SimulationControl.createApp(new EEGSimulatorApp());
}
}

```

EEGSimulatorApp2.java

```
package org.opensourcephysics.sip.ch10;

import java.io.*;
import java.util.*;

public class EEGSimulator2App {
    static int gridSize = 16;
    static int numSamples = 100;
    static double[][] potential;
    static boolean[][] isConductor;
    static int maxIterations = 1000;
    static double tolerance = 0.01;

    static double[][] electrodeData = new double[numSamples][4];
    static int[][] dipoleCoords = new int[numSamples][4];

    public static void main(String[] args) throws IOException {
        Random rand = new Random();

        for (int sample = 0; sample < numSamples; sample++) {
            potential = new double[gridSize][gridSize];
            isConductor = new boolean[gridSize][gridSize];

            // Generate valid dipole location (avoid edges and corners)
            int px, py, nx, ny;
            do {
                px = 1 + rand.nextInt(gridSize - 2);
                py = 1 + rand.nextInt(gridSize - 2);

                int[][] options = {{1,0},{-1,0},{0,1},{0,-1}};
                int[] offset = options[rand.nextInt(4)];
                nx = px + offset[0];
                ny = py + offset[1];
            } while (onEdgeOrCorner(px, py) || onEdgeOrCorner(nx, ny));

            // Apply dipole
            potential[px][py] = 100;
            potential[nx][ny] = -100;
            isConductor[px][py] = true;
            isConductor[nx][ny] = true;

            // Relax
```

```

    relax();

    // Store corner values
    electrodeData[sample][0] = potential[1][1];
    electrodeData[sample][1] = potential[1][gridSize - 2];
    electrodeData[sample][2] = potential[gridSize - 2][1];
    electrodeData[sample][3] = potential[gridSize - 2][gridSize - 2];

    dipoleCoords[sample][0] = px;
    dipoleCoords[sample][1] = py;
    dipoleCoords[sample][2] = nx;
    dipoleCoords[sample][3] = ny;
}

// Save to Desktop
String desktopPath = System.getProperty("user.home") + File.separator + "Desktop";
String fullPath = desktopPath + File.separator + "dipole_data.csv";
saveCSV(fullPath);
System.out.println("CSV file created at: " + fullPath);
}

public static boolean onEdgeOrCorner(int x, int y) {
    return (x <= 0 || y <= 0 || x >= gridSize - 1 || y >= gridSize - 1);
}

public static void relax() {
    for (int iter = 0; iter < maxIterations; iter++) {
        double maxDiff = 0;
        double[][] newPotential = new double[gridSize][gridSize];

        for (int i = 1; i < gridSize - 1; i++) {
            for (int j = 1; j < gridSize - 1; j++) {
                if (isConductor[i][j]) {
                    newPotential[i][j] = potential[i][j];
                } else {
                    newPotential[i][j] = 0.25 * (potential[i+1][j] + potential[i-1][j] +
                        potential[i][j+1] + potential[i][j-1]);
                    maxDiff = Math.max(maxDiff, Math.abs(newPotential[i][j] - potential[i][j]));
                }
            }
        }
        potential = newPotential;
        if (maxDiff < tolerance) break;
    }
}

```

```

    }

    public static void saveCSV(String filepath) throws IOException {
        BufferedWriter writer = new BufferedWriter(new FileWriter(filepath));
        writer.write("UL,UR,LL,LR,pos_x,pos_y,neg_x,neg_y\n");
        for (int i = 0; i < numSamples; i++) {
            writer.write(electrodeData[i][0] + "," +
                electrodeData[i][1] + "," +
                electrodeData[i][2] + "," +
                electrodeData[i][3] + "," +
                dipoleCoords[i][0] + "," +
                dipoleCoords[i][1] + "," +
                dipoleCoords[i][2] + "," +
                dipoleCoords[i][3] + "\n");
        }
        writer.close();
    }
}

```

predictdipole.py

```
!pip install -q seaborn scikit-learn pandas matplotlib
```

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.multioutput import MultiOutputRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
uploaded = files.upload()

data = pd.read_csv("dipole_data.csv")

X = data[["UL", "UR", "LL", "LR"]]
y = data[["pos_x", "pos_y", "neg_x", "neg_y"]]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```



```

model = MultiOutputRegressor(RandomForestRegressor(n_estimators=100, random_state=42))
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

grid_size = 16
y_pred_rounded = np.round(y_pred).astype(int)
y_pred_rounded = np.clip(y_pred_rounded, 0, grid_size - 1)
y_true = y_test.values.astype(int)

exact_match = np.all(y_pred_rounded == y_true, axis=1)
exact_accuracy = np.mean(exact_match)
distance_errors = np.sqrt(np.sum((y_pred_rounded - y_true) ** 2, axis=1))
mean_distance_error = np.mean(distance_errors)

results = pd.DataFrame({
    "True_Pos_X": y_true[:, 0], "Pred_Pos_X": y_pred_rounded[:, 0],
    "True_Pos_Y": y_true[:, 1], "Pred_Pos_Y": y_pred_rounded[:, 1],
    "True_Neg_X": y_true[:, 2], "Pred_Neg_X": y_pred_rounded[:, 2],
    "True_Neg_Y": y_true[:, 3], "Pred_Neg_Y": y_pred_rounded[:, 3],
})

print("Exact Match Accuracy: {:.2f}%".format(exact_accuracy * 100))
print("Mean Euclidean Distance Error: {:.2f} grid units\n".format(mean_distance_error))
print("Sample Predictions:\n")
print(results.head())

heatmap_pos = np.zeros((grid_size, grid_size), dtype=int)
for x, y in y_pred_rounded[:, :2]:
    heatmap_pos[y, x] += 1

plt.figure(figsize=(6, 5))
sns.heatmap(heatmap_pos, cmap="viridis", square=True, cbar_kws={'label': 'Prediction Count'})
plt.title("Heatmap of Predicted Positive Dipole Positions")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.tight_layout()
plt.show()

```